GPU Memory Calculation and Configuration

This guide explains how to calculate GPU memory requirements and properly configure `gpu_memory_utilization` for vLLM-Omni stages.

Overview

`gpu_memory_utilization` is a critical parameter that controls how much GPU memory each stage can use. It's specified as a fraction between 0.0 and 1.0, where: - `0.8` means 80% of the GPU's total memory - `1.0` means 100% of the GPU's total memory (not recommended, leaves no buffer)

How Memory is Calculated

Memory Allocation Formula

For each stage, vLLM-Omni calculates the requested memory as:

```
requested_memory = total_gpu_memory × gpu_memory_utilization
```

The system checks that:

```
free_memory ≥ requested_memory
```

If this condition is not met, the stage will fail to initialize with an error message showing the memory requirements.

Memory Components

The total memory used by a stage includes:

1. **Model Weights**: The size of the model parameters loaded on the GPU
2. **KV Cache**: Memory for storing key-value cache during generation
3. **Activation Memory**: Temporary memory for intermediate computation
4. **System Overhead**: Memory used by CUDA, PyTorch, and other system components

stable

5. **Non-Torch Memory**: Memory allocated outside of PyTorch (e.g., CUDA graphs)

Example Calculation

For a GPU with 80GB total memory: - `gpu_memory_utilization: 0.8` → 64GB available for the stage - `gpu_memory_utilization: 0.6` → 48GB available for the stage - `gpu_memory_utilization: 0.15` → 12GB available for the stage

Setting Up `gpu_memory_utilization`

Step 1: Determine GPU Memory

First, check your GPU's total memory:

```
# Using nvidia-smi
nvidia-smi --query-gpu=memory.total --format=csv

# Or using Python
python -c "import torch; print(f'{torch.cuda.get_device_properties(0).total_memory / 1e9:.1f} GB')"
```

Step 2: Estimate Model Memory Requirements

For Autoregressive (AR) Stages

AR stages typically need more memory due to: - Large model weights - KV cache for attention - Activation buffers

For Diffusion/Generation Stages

Diffusion stages (like code2wav) typically need less memory: - Smaller model components - Different memory access patterns

Typical values: - `0.1 - 0.3` for most diffusion stages

Step 3: Consider Multi-Stage Scenarios

When multiple stages share the same GPU, you must ensure the sum of their `gpu_memory_utilization` values doesn't exceed 1.0.

stable

Example: Two stages on GPU 0

```yaml
stage_args:
  - stage_id: 0
    runtime:
      devices: "0"
    engine_args:
      gpu_memory_utilization: 0.6  # Uses 60% of GPU 0

  - stage_id: 1
    runtime:
      devices: "0"
    engine_args:
      gpu_memory_utilization: 0.3  # Uses 30% of GPU 0
      # Total: 90% of GPU 0 (safe, leaves 10% buffer)
```

Important: If stages run on different GPUs, each can use up to 1.0 independently.

Step 4: Account for Tensor Parallelism

When using `tensor_parallel_size > 1`, the model is split across multiple GPUs, so each GPU needs less memory.

Example: 2-way tensor parallelism

```yaml
stage_args:
  - stage_id: 0
    runtime:
      devices: "0,1"  # Uses both GPUs
    engine_args:
      tensor_parallel_size: 2
      gpu_memory_utilization: 0.6  # 60% per GPU
      # Model is split, so each GPU uses ~30% of model memory
```

Examples

Qwen3-Omni-MoE on 2x H100-80GB

```yaml
stage_args:
  - stage_id: 0  # Thinker stage with TP=2
    runtime:
      devices: "0,1"
    engine_args:
      tensor_parallel_size: 2
      gpu_memory_utilization: 0.6  # 48GB per GPU

  - stage_id: 1  # Talker stage
    runtime:
      devices: "1"
```

stable

```
    engine_args:
      gpu_memory_utilization: 0.3  # 24GB on GPU 1

  - stage_id: 2  # Code2Wav stage
    runtime:
      devices: "0"
    engine_args:
      gpu_memory_utilization: 0.1  # 8GB on GPU 0
```

Note: In this configuration, stages 0 and 2 share GPU 0, but they run at different times in the pipeline, so their memory usage doesn't overlap.

Troubleshooting

Error: "Free memory is less than desired GPU memory utilization"

This means the GPU doesn't have enough free memory when the stage starts.

Solutions: 1. Free up memory by closing other processes 2. Reduce `gpu_memory_utilization` for this stage 3. Use a GPU with more memory 4. Move the stage to a different GPU

Error: OOM during inference

The stage initialized but ran out of memory during processing.

Solutions: 1. Reduce `max_num_batched_tokens` 2. Reduce `max_num_seqs` in engine_args 3. Lower `gpu_memory_utilization` slightly 4. Enable quantization if supported

Memory Not Fully Utilized

If you see low memory usage, you can: 1. Increase `gpu_memory_utilization` to allow larger KV cache 2. Increase `max_num_batched_tokens` for better batching 3. Check if other stages are limiting throughput

Useful formula for Memory Calculation

KV Cache Memory

The KV cache size depends on: - Number of sequences in batch - Sequenc stable generation) - Model hidden size - Number of attention heads - Number of layers

approximate Formula:

```
kv_cache_memory ≈ batch_size × seq_len × hidden_size × num_layers × 2 × dtype_size
```

2 for k & v

Model Weight Memory

```
model_memory ≈ num_parameters × dtype_size
```

For example: - 7B parameters in FP16: ~14GB - 7B parameters in FP32: ~28GB - 7B parameters in INT8: ~7GB

Activation Memory

Activation memory is typically smaller but varies with: - Batch size - Sequence length - Model architecture

It's usually 10-30% of model weight memory during inference.

March 20, 2026

stable